



ios

iOS Development



→ iPadOS

- iOS is Apple's mobile operating system for iPhone, iPad, and ~~iPod Touch~~ - deprecated
- iOS development focuses on building high-performance, secure, and user-friendly apps
- Primary ecosystem: Apple devices only
- App distribution via Apple App Store
- Ecosystem
 - Devices: iPhone, iPad, Apple Watch, Apple TV
 - Operating Systems: iOS, iPadOS, watchOS, tvOS, visionOS
 - App Store, TestFlight, App Store Connect
 - Apple Developer Program - \$99 / year
- Development Tools
 - Xcode - Official IDE for iOS development
 - Interface Builder / SwiftUI Preview
 - Simulator for device testing
 - Instruments for performance analysis

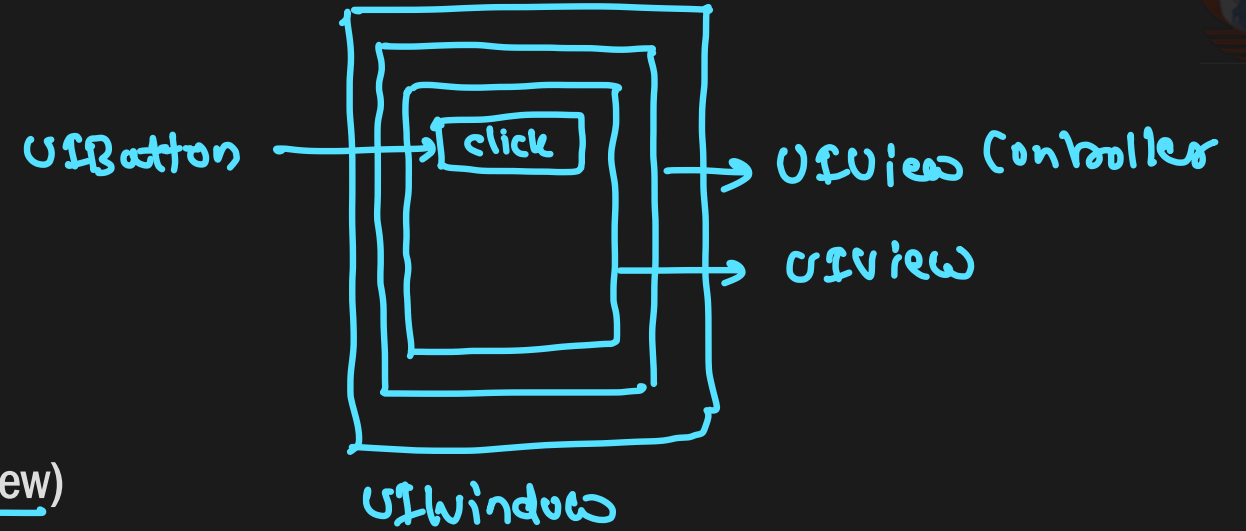
UIKit

Framework = collection of swift entities (classes | structs | enums | functions)

- UIKit is Apple's original (and still primary) framework for building graphical, event-driven user interfaces on iOS and iPadOS
- UIKit provides every visual element and system interaction you see in an iPhone app
- It includes
 - Screen Elements
 - UIButton, UILabel, UITextField, UIImageView, UISlider (your buttons, text, images).
 - Containers
 - UITableView, UICollectionView (lists and grids), and—most importantly—UIViewController (which we just discussed).
 - Navigation
 - UINavigationController, UITabBarController (the back buttons and tab bars at the bottom).
 - User Interaction
 - Gesture recognizers (UITapGestureRecognizer), touch handling, and animation engines.
 - System Integration
 - Keyboards, device rotation, screen brightness, haptic feedback (vibration), and accessibility (VoiceOver).

View Hierarchy

- UIApplication
 - UIWindow (The very top-level container)
 - UIViewController (The manager)
 - UIView (The root view of the VC)
 - Subview: UIStackView (A container view)
 - Subview: UILabel (A view that draws text)
 - Subview: UIImageView (A view that draws a photo)
 - Subview: UIButton (A view that draws a button and handles taps)
 - Subview: UILabel (The button's title text)



UIApplication



- UIApplication is a singleton object (meaning there is only one in your entire app)
- You access it via UIApplication.shared
- Its job is to manage the event loop (the constant waiting for taps, swipes, and system notifications) and to hand off crucial system-level messages to your App Delegate
- The UIApplication handles the phone's commands
- The App Delegate is your code's response to those commands

Application lifecycle

State	What it means
<u>Not Running</u>	<u>The app hasn't been launched yet, or it was terminated by iOS or the user.</u>
<u>Inactive</u>	<u>The app is launching or about to be suspended. It is in the foreground but not receiving events (e.g., a phone call interrupts it, or you pull down the Control Center).</u>
<u>Active</u>	<u>The normal state. The app is in the foreground and actively receiving user events (taps, swipes, typing).</u>
<u>Background</u>	<u>The app is in the background, but still executing code (usually for a few seconds to finish a network request).</u>
<u>Suspended</u>	<u>The app is in the background and frozen in memory. It is not executing any code. iOS can evict it from memory at any time to free up RAM for the foreground app.</u>

→ app is installed but not started

UIView



- UIView is the fundamental building block of your app's user interface
- Everything you see on an iPhone screen—a button, a label, a text field, a slider, or even the blank white background—is a UIView (or a subclass of it)
- Core properties of UIView
 - ✓ **Frame & Bounds:** Its position (x, y) and size (width, height) on the screen.
 - ✓ **Background Color:** The color filling its rectangular area.
 - ✓ **Alpha:** Its transparency (0.0 = invisible, 1.0 = fully opaque).
 - ✓ **Hidden:** Whether it is visible or not.
 - ✓ **Layer:** A CALayer that handles the actual drawing, shadows, rounded corners, and borders. (Every view has one).
 - ✓ **Tag:** A number you can assign to easily find the view later in your code.

UIView most common subclasses



- UILabel – Displays static text.
- UIButton – Displays text/image and responds to taps.
- UIImageView – Displays a picture.
- UITextField – Allows the user to type a single line of text.
- UITextView – Allows the user to type multi-line scrollable text.
- UIScrollView – A view that allows zooming and panning around content larger than the screen.
- UITableView – Displays a scrollable list of rows (Reuses views for performance).
- UICollectionView – Displays a scrollable grid of items.
- UISwitch – The classic on/off toggle.
- UIStackView – A container view that automatically arranges its children horizontally or vertically (saves you from writing tons of layout code).

UIViewController → Activity in Android



- In Apple's Model-View-Controller (MVC) architecture, the View Controller is the "C", the glue that holds everything together
- View = What the user sees (buttons, labels, images)
- Model = Your data (user profiles, scores, settings)
- Controller = The traffic cop. It listens to user taps, tells the Model to update, and refreshes the View to display the new data.
- A View Controller manages a single "screen" (or a portion of a screen) of your app. It owns the content and dictates how that content reacts to the user
- **Responsibilities**
 - Managing the View Hierarchy: Adding, removing, and arranging subviews (usually done via Storyboards or Auto Layout constraints).
 - Handling User Interactions: Connecting @IBAction methods from UI elements (like buttons) to code.
 - Data Management: Preparing data to be displayed (e.g., fetching an array of names and giving it to a UITableView).
 - Navigation: Pushing, presenting, or dismissing other View Controllers (e.g., navigationController?.pushViewController()).
 - Coordinating with Models: Making network requests, reading from a database, and passing that raw data into the views.

Types of UIViewController



- Apple divides VCs into two categories:
- **Content View Controllers:** These manage a self-contained screen of content
 - UIViewController (generic)
 - UITableViewController (manages a table view specifically)
 - UICollectionViewController (manages a collection grid)
- **Container View Controllers:** These do not have their own content. Instead, they manage *other* View Controllers and coordinate their layout
 - UINavigationController (manages a stack of VCs, providing a back button and title bar)
 - UITabBarController (manages a tabbed interface)
 - UISplitViewController (manages master/detail layouts for iPads)
 - UIPageViewController (manages swiping between pages)

Life Cycle of UIViewController



- init(coder:) / init(nibName:bundle:)
 - The VC is born in memory (instantiated).
- viewDidLoad() → onCreate()
 - The most important one
 - Called once after the view is loaded into memory
 - This is where you set up initial data, register table view cells, and configure the UI elements
 - The views' frames have not been finalized yet, so do not compute layout here ✖✖
- viewWillAppear(_:) → onStart()
 - Called right before the screen appears on the device
 - Used to start animations, begin network calls, or refresh data that might have changed since the last time the user saw this screen
- viewDidAppear(_:) → onResume()
 - Called right after the screen appears
 - Used to start timers, start observing notifications, or start analytics tracking (since the user can now see the screen)
- viewWillDisappear(_:) → onPause()
 - Called right before the screen is covered or dismissed. Used to save user data or stop an audio player
- viewDidDisappear(_:) → onStop()
 - Called right after the screen vanishes. Used to stop timers, remove observers, or cancel network requests

